

# Дизайн и анализ на алгоритми упражнение 5

Станислав Игов (Стъката)

2025/2026

## LongestUniqueSubarray. Разделяй и владей. Binary Search. Flip, Мас-тър теорема

### LongestUniqueSubarray

**Вход:** Масив  $A[1..n]$  от цели неотрицателни числа, т.ч.  $A[i] \leq n$  за  $i \in [1..n]$

**Изход:** Дължината на най-дългия инфикс с уникални елементи, тоест  $\max_{1 \leq p \leq q \leq n} \{q - p + 1 \mid A[p..q] \text{ има уникални елементи}\}$

LongestUniqueSubarray( $A[1..n]$ : array of unsigned int)

```
@1 B[0..n] <- malloc(n+1){0..0}
@2 l <- 1
@3 best <- 0
@4 for r <- 1 to n do
@5     while B[A[r]] = 1 do
@6         B[A[l]] <- 0
@7         l++
@8     endwhile
@9     B[A[r]] <- 1
@10    best <- max(best, r - l + 1)
@11 endfor
@12 return best
```

Нека  $l_i$ ,  $best_i$ ,  $B_i$  са стойностите на  $l$ ,  $best$  и масива  $B$  преди  $i$ -тата итерация на цикъла на ред @4. Дефинираме инварианта на външния цикъл:

$$I_{out}(i) \iff \begin{cases} best_i = \max_{1 \leq p \leq q < i} \{q - p + 1 \mid A[p..q] \text{ има уникални елементи}\} \\ A[l_i..i-1] \text{ има уникални елементи} \\ B_i[v] = 1 \iff v \in \{A[l_i], A[l_i+1], \dots, A[i-1]\} \end{cases}$$

Тоест:

- $best_i$  е дължината на най-дългия уникален инфикс, разгледан до момента.
- Текущият прозорец  $[l_i, i-1]$  съдържа само уникални елементи.
- $B_i$  отразява точно кои стойности се намират в текущия прозорец.

### Доказателство

#### База

Преди първата итерация имаме  $l_1 = 1$ ,  $best_1 = 0$ ,  $B_1 = \{0, \dots, 0\}$ .

Прозорецът  $A[1..0]$  е празен, следователно тривиално има уникални елементи.  $B_1$  е нулев масив, което правилно отразява, че прозорецът е празен.  $best_1 = 0$ , което е коректно тъй като не сме разгледали нито един инфикс.

Следователно  $I_{out}(1)$  е вярно.

## Поддръжка

Допускаме, че  $I_{out}(i)$  е вярно. Ще докажем, че след изпълнението на  $i$ -тата итерация е вярно  $I_{out}(i+1)$ .

При влизане в итерацията имаме  $r = i$ . Преди ред 05 прозорецът  $A[l_i..i-1]$  има уникални елементи и  $B_i$  го отразява коректно.

Трябва да добавим  $A[r] = A[i]$  към прозореца. Разглеждаме вътрешния цикъл на ред 05.

Нека  $l^{(t)}$  и  $B^{(t)}$  са състоянията на  $l$  и  $B$  преди  $t$ -тата итерация на вътрешния цикъл.

Дефинираме инварианта на вътрешния цикъл:

$$I_{in}(t) \iff \begin{cases} A[l^{(t)}..i-1] \text{ има уникални елементи} \\ B^{(t)}[v] = 1 \iff v \in \{A[l^{(t)}], \dots, A[i-1]\} \end{cases}$$

Тоест прозорецът  $[l^{(t)}, i-1]$  остава уникален и  $B^{(t)}$  го отразява коректно след всяко свиване от ляво.

**База на вътрешния инвариант.** Преди първата итерация на вътрешния цикъл имаме  $l^{(1)} = l_i$ . По  $I_{out}(i)$  прозорецът  $A[l_i..i-1]$  има уникални елементи и  $B_i$  го отразява коректно. Следователно  $I_{in}(1)$  е вярно.

**Поддръжка на вътрешния инвариант.** Допускаме, че  $I_{in}(t)$  е вярно и условието на ред 05 е истина, т.е.  $B[A[i]] = 1$ , което означава  $A[i] \in \{A[l^{(t)}], \dots, A[i-1]\}$ .

На редове 06–07 се изпълнява

$$B^{(t)}[A[l^{(t)}]] \leftarrow 0, \quad l^{(t+1)} \leftarrow l^{(t)} + 1.$$

Следователно  $l^{(t+1)} = l^{(t)} + 1$  и  $B^{(t+1)}$  вече не съдържа  $A[l^{(t)}]$ . Прозорецът  $A[l^{(t+1)}..i-1]$  е подмасив на  $A[l^{(t)}..i-1]$ , следователно също има уникални елементи и  $B^{(t+1)}$  го отразява коректно.

Следователно  $I_{in}(t+1)$  е вярно.

**Терминация на вътрешния цикъл.** Вътрешният цикъл приключва когато  $B[A[i]] = 0$ , т.е.  $A[i] \notin \{A[l^{(t)}], \dots, A[i-1]\}$ .

По  $I_{in}(t)$  прозорецът  $A[l^{(t)}..i-1]$  има уникални елементи и  $A[i]$  не се среща в него. Следователно след ред 09:

$$B[A[i]] \leftarrow 1$$

прозорецът  $A[l^{(t)}..i]$  има уникални елементи и  $B$  го отразява коректно.

Нека  $l_{i+1} = l^{(t)}$  е стойността на  $l$  след края на вътрешния цикъл. На ред 010:

$$best_{i+1} \leftarrow \max(best_i, i - l_{i+1} + 1)$$

По  $I_{out}(i)$   $best_i$  е максималната дължина за инфиксите в  $A[1..i-1]$ . Новият прозорец  $A[l_{i+1}..i]$  е най-дългият уникален инфикс завършващ на позиция  $i$ , защото вътрешният цикъл свива от ляво само колкото е необходимо. Следователно  $best_{i+1}$  е максималната дължина за инфиксите в  $A[1..i]$ .

Следователно  $I_{out}(i+1)$  е вярно.

## Терминация

Външният цикъл приключва когато  $r = n+1$ , т.е. след  $n$ -тата итерация. По  $I_{out}(n+1)$ :

$$best_{n+1} = \max_{1 \leq p \leq q \leq n} \{q - p + 1 \mid A[p..q] \text{ има уникални елементи}\}$$

Следователно алгоритъмът връща коректния отговор и е доказан.

## Времева сложност

### Очевиден анализ

На пръв поглед алгоритъмът има два вложени цикъла — външен **for** и вътрешен **while**. Ако оценим всяка итерация на вътрешния цикъл като  $O(1)$ , и допуснем, че той се изпълнява  $O(n)$  пъти за всяка итерация на външния, получаваме горна граница  $O(n^2)$ .

## Амортизиран анализ

По-задълбоченият анализ показва, че истинската сложност е  $O(n)$ .

Ключовото наблюдение е следното: разглеждаме броя пъти, които всеки индекс може да бъде “добавен” и “премахнат” от прозореца.

- Всеки индекс  $i \in [1..n]$  се добавя към прозореца **точно веднъж** — когато  $r = i$  на ред ⑨.
- Всеки индекс  $i \in [1..n]$  се премахва от прозореца **най-много веднъж** — когато  $l$  премине през него на ред ⑦.

Следователно общият брой операции извършени от вътрешния цикъл през **цялото** изгълнение на алгоритъма е най-много  $n$ , а не  $O(n)$  за всяка итерация на външния.

Следователно  $\text{Time}(\text{LongestUniqueSubarray}) \in O(n)$ .

## Разделяй и владей

Парадигмата *разделяй и владей* се основава на три стъпки:

- **Разделяй** — разбий задачата на по-малки подзадачи от същия вид.
- **Владей** — реши подзадачите рекурсивно.
- **Комбинирай** — обедини решенията на подзадачите в решение на оригиналната задача.

## Двоично търсене (Binary Search)

**Вход:** Масив  $A[1..n]$  от рационални числа, сортиран във възходящ ред,  $target$  търсена стойност.

**Изход:** Индекс  $i$  т.ч.  $A[i] = target$ , или  $-1$  ако  $target$  не се среща в  $A$ .

```
BinarySearch(A[1..r]: array of rationals, target: rational)
①  if l > r then
②      return -1
③  endif
④  mid <- (l + r) / 2 //ако числото не е цяло закръгляваме надолу
⑤  if A[mid] = target then
⑥      return mid
⑦  else if A[mid] < target then
⑧      return BinarySearch(A[mid+1..r], target)
⑨  else
⑩      return BinarySearch(A[1..mid-1], target)
⑪  endif
```

## Доказателство за коректност

Ще докажем с индукция по размера на входа  $n = r - l + 1$ , че ако  $target$  се намира в  $A[l..r]$ , алгоритъмът ще го намери.

**База:**  $n = 0$

Тогава  $l > r$  и условието на ред ① е истина. Алгоритъмът връща  $-1$ . Тъй като масивът е празен,  $target$  не се среща в него, следователно  $-1$  е коректен отговор.

**База:**  $n = 1$

Тогава  $l = r$  и  $mid = l$ .

- Ако  $A[mid] = target$  — алгоритъмът връща  $mid$ . Коректно.
- Ако  $A[mid] \neq target$  — рекурсивното извикване е върху масив с  $n = 0$  елемента, което е коректно по предната база.

## Индукционна стъпка

Допускаме, че алгоритъмът е коректен за **всички** масиви с размер  $< n$ . Ще докажем, че е коректен и за масив с размер  $n > 1$ .

Изчисляваме  $mid = \lfloor (l + r)/2 \rfloor$ .

**Случай 1:**  $A[mid] = target$ . Алгоритъмът връща  $mid$ . Коректно.

**Случай 2:**  $A[mid] < target$ . Тъй като масивът е сортиран,  $target$  може да се намира само в  $A[mid + 1..r]$ . Подмасивът  $A[mid + 1..r]$  има размер  $r - mid < n$ . По индукционната хипотеза `BinarySearch`( $A[mid + 1..r], target$ ) ще намери  $target$  ако се среща там, и ще върне  $-1$  в противен случай. Следователно резултатът е коректен.

**Случай 3:**  $A[mid] > target$ . Аналогично  $target$  може да се намира само в  $A[l..mid - 1]$ , който има размер  $mid - l < n$ . По индукционната хипотеза резултатът е коректен.

Следователно алгоритъмът `BinarySearch` е коректен.

## Времева сложност

Записваме рекурентното уравнение:

$$T(n) = T\left(\frac{n}{2}\right) + O(1), \quad T(1) = O(1), \quad T(0) = O(1)$$

## Решаване чрез разгъване

Разгъваме рекурентното уравнение:

$$T(n) = T\left(\frac{n}{2}\right) + c = T\left(\frac{n}{4}\right) + c + c = T\left(\frac{n}{8}\right) + 3c = \dots = T\left(\frac{n}{2^k}\right) + kc$$

Достигахме базовия случай когато  $\frac{n}{2^k} = 1$ , т.е.  $k = \log_2 n$ .

Следователно:

$$T(n) = T(1) + c \log_2 n = O(1) + O(\log n) = O(\log n)$$

$\text{Time}(\text{BinarySearch}) \in O(\log n)$ .

## Задача Flip

**Вход:** Масив  $A[1..n]$  от рационални числа т.ч.  $A[1] > A[n]$ .

**Изход:** Индекс  $i$ , т.ч.  $1 \leq i < n$  и  $A[i] > A[i + 1]$ .

## Съществуване на решение

**Лема:** За всеки масив  $A[1..n]$  т.ч.  $A[1] > A[n]$ , съществува индекс  $i \in [1, n - 1]$  т.ч.  $A[i] > A[i + 1]$ .

**Доказателство:** Допускаме обратното тоест  $A[i] \leq A[i + 1]$  за всяко  $i \in [1, n - 1]$ . Тогава редицата е нарастваща и  $A[1] \leq A[n]$ , което противоречи на условието  $A[1] > A[n]$ . Следователно такъв индекс съществува.

## Алгоритъм

`Flip(A[1..r]): array of rationals`

```
@1  if l + 1 = r then
@2      return l
@3  endif
@4  mid <- (l + r) / 2
@5  if A[mid] > A[mid+1] then
@6      return mid
@7  else if A[l] > A[mid] then
```

```

@8      return Flip(A[l..mid])
@9  else
@10     return Flip(A[mid..r])
@11 endif

```

## Доказателство за коректност

Доказателството е аналогично на това за **BinarySearch** — индукция по размера на входа  $n = r - l + 1$ .

Ключовото наблюдение е, че на всяка стъпка запазваме инварианта  $A[l] > A[r]$ , следователно по Лемата решение винаги съществува в текущия подмасив.

- **База:** При  $l + 1 = r$  имаме  $A[l] > A[r]$ , следователно  $A[l] > A[l + 1]$  и  $l$  е коректен отговор.
- **Случай 1**  $A[mid] > A[mid + 1]$ : Намерихме директно търсения индекс.
- **Случай 2**  $A[l] > A[mid]$ : Инвариантът  $A[l] > A[mid]$  е изпълнен, следователно решение съществува в  $A[l..mid]$ . Рекурсивното извикване е върху подмасив с размер  $< n$  и по индукционната хипотеза е коректно.
- **Случай 3**  $A[mid] > A[r]$ : Тъй като  $A[mid] \leq A[mid + 1]$  и  $A[l] \leq A[mid]$ , следва  $A[mid] \geq A[l] > A[r]$ , следователно инвариантът е изпълнен за  $A[mid..r]$ . Рекурсивното извикване е коректно по индукционната хипотеза.

Следователно алгоритъмът **Flip** е коректен.

## Времева сложност

Рекурентното уравнение е идентично с това на **BinarySearch**:

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

Следователно  $\text{Time}(\text{Flip}) \in O(\log n)$ .

## Мастър теорема

Нека е дадено рекурентно уравнение от вида:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

където  $a \geq 1$ ,  $b > 1$  са константи и  $f(n)$  е асимптотично положителна функция.

Дефинираме  $d = \log_b a$ .

- **Случай 1:** Ако  $f(n) \in O(n^{d-\varepsilon})$  за някое  $\varepsilon > 0$ , то

$$T(n) \in \Theta(n^d)$$

- **Случай 2:** Ако  $f(n) \in \Theta(n^d)$ , то

$$T(n) \in \Theta(n^d \log n)$$

- **Случай 3:** Ако  $f(n) \in \Omega(n^{d+\varepsilon})$  за някое  $\varepsilon > 0$  и  $a \cdot f(n/b) \leq c \cdot f(n)$  за някоя константа  $c < 1$ , то

$$T(n) \in \Theta(f(n))$$

## Примери

**Пример 1: BinarySearch.**

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

Тук  $a = 1$ ,  $b = 2$ ,  $f(n) = O(1)$ ,  $d = \log_2 1 = 0$ .

$f(n) = O(1) = \Theta(n^0) = \Theta(n^d)$  — Случай 2.

$$T(n) \in \Theta(\log n)$$

**Пример 2: MergeSort.**

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Тук  $a = 2$ ,  $b = 2$ ,  $f(n) = O(n)$ ,  $d = \log_2 2 = 1$ .  
 $f(n) = \Theta(n^1) = \Theta(n^d)$  — Случай 2.

$$T(n) \in \Theta(n \log n)$$

**Пример 3.**

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

Тук  $a = 4$ ,  $b = 2$ ,  $f(n) = O(n)$ ,  $d = \log_2 4 = 2$ .  
 $f(n) = O(n) = O(n^{2-1}) = O(n^{d-\varepsilon})$  за  $\varepsilon = 1$  — Случай 1.

$$T(n) \in \Theta(n^2)$$

**Пример 4.**

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$$

Тук  $a = 2$ ,  $b = 2$ ,  $f(n) = O(n^2)$ ,  $d = \log_2 2 = 1$ .  
 $f(n) = O(n^2)$  което расте по-бързо от  $O(n) \implies$   
 $f(n) = \Omega(n^2) = \Omega(n^{1+1}) = \Omega(n^{d+\varepsilon})$  за  $\varepsilon = 1$  Случай 3.  
Проверяваме регулярното условие:  $2 \cdot (n/2)^2 = n^2/2 \leq c \cdot n^2$  за  $c = 1/2 < 1$ .

$$T(n) \in \Theta(n^2)$$

**Пример 5.**

$$T(n) = 3T\left(\frac{n}{3}\right) + O(n)$$

Тук  $a = 3$ ,  $b = 3$ ,  $f(n) = O(n)$ ,  $d = \log_3 3 = 1$ .  
 $f(n) = \Theta(n^1) = \Theta(n^d)$  — Случай 2.

$$T(n) \in \Theta(n \log n)$$

**Пример при който Мастър теоремата не е приложима**

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n \log n)$$

Тук  $a = 2$ ,  $b = 2$ ,  $d = \log_2 2 = 1$ .  
 $f(n) = n \log n$  не попада в нито един от трите случая —  $f(n)$  е асимптотично по-голямо от  $\Theta(n^1)$ , но не е полиномиално по-голямо, тъй като  $\log n \in o(n^\varepsilon)$  за всяко  $\varepsilon > 0$ .  
Следователно Мастър теоремата не е приложима и трябва да използваме разгъване.